

Day 1 · Topic 1: HuggingFace Transformers

Week 1 — Local Inferencing Foundations

Goal: Run your first local LLM end-to-end using the HuggingFace Transformers library. Understand the difference between the high-level *pipeline()* API and the low-level *AutoModel* API, and learn how text generation works on CPU.

1. Technical Explanation

What is HuggingFace Transformers?

The **transformers** library (by Hugging Face) is the de-facto Python SDK for loading, running, and fine-tuning transformer-based language models. It abstracts away the differences between model architectures (GPT-2, LLaMA, Mistral, Falcon ...) behind a consistent API, and handles weight downloading, tokenization, and device placement automatically.

The Two API Layers

Aspect	<code>pipeline()</code>	<code>AutoModel</code> / <code>AutoTokenizer</code>
Abstraction level	High — one-liner	Low — full control
Use case	Prototyping, quick tests	Custom loops, fine-tuning, batching
Device control	<code>device=0</code> shortcut	<code>model.to('cuda')</code> explicit
Output format	Dict with decoded text	Raw logits / hidden states
Token management	Hidden internally	Explicit encode/decode

How Text Generation Works (Autoregressive Decoding)

LLMs generate text one **token** at a time. At each step the model ingests all tokens seen so far and produces a probability distribution over its entire vocabulary. A decoding strategy (greedy, top-k, top-p / nucleus, beam search) picks the next token. That token is appended to the context and the loop repeats until an EOS token is produced or *max_new_tokens* is reached.

Memory & VRAM Mechanics (CPU path)

On CPU, all model weights and activation tensors live in system RAM. A 7B-parameter model in FP32 requires roughly **28 GB** of RAM (7×10^9 params $\times$ 4 bytes). Loading in FP16 halves that to ~14 GB. For Day 1 we start small (GPT-2, ~500 MB) so any modern laptop can run it. Token generation is purely sequential on CPU — expect 1–5 tokens/sec depending on model size and hardware.

Tip: Always start with a small model (GPT-2 or distilgpt2) when learning the API. Once the workflow is solid, scale up to 7B models on GPU.

2. Practical Implementation

2.1 Installation

```
# Create a clean virtual environment first (recommended)
python -m venv llm-env
source llm-env/bin/activate          # Windows: llm-env\Scripts\activate

# Install core packages
pip install transformers torch accelerate sentencepiece
# For fast tokenizers (Rust-based)
pip install tokenizers
```

2.2 High-Level API — pipeline()

```
from transformers import pipeline
import time

# pipeline() downloads weights on first run (~500 MB for gpt2)
generator = pipeline(
    task="text-generation",
    model="gpt2",          # swap to "openai-community/gpt2-medium" for 345M
    device=-1              # -1 = CPU | 0 = first GPU
)

prompt = "The future of AI infrastructure is"

start = time.perf_counter()
outputs = generator(
    prompt,
    max_new_tokens=80,
    do_sample=True,        # enable stochastic sampling
    temperature=0.8,       # diversity control (lower = more focused)
    top_p=0.92,            # nucleus sampling
    num_return_sequences=1,
)
elapsed = time.perf_counter() - start

text = outputs[0]["generated_text"]
new_tokens = len(text.split()) - len(prompt.split()) # rough estimate

print("=== Generated Text ===")
print(text)
print(f"\nElapsed      : {elapsed:.2f}s")
print(f"New tokens      : ~{new_tokens}")
print(f"Tok/sec         : ~{new_tokens / elapsed:.1f}")
```

2.3 Low-Level API — AutoModel + AutoTokenizer

```
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```

import torch
import time

MODEL_ID = "gpt2" # change to any causal LM on HuggingFace Hub

# Step 1: Load tokenizer (converts text <-> token IDs)
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)

# Step 2: Load model weights
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.float32, # use float16 if you have a GPU
)
model.eval() # disable dropout for inference

# Step 3: Tokenize prompt
prompt = "The future of AI infrastructure is"
inputs = tokenizer(prompt, return_tensors="pt")
print(f"Input token IDs : {inputs['input_ids']}")
print(f"Token count      : {inputs['input_ids'].shape[1]}")

# Step 4: Greedy generation (no sampling)
start = time.perf_counter()
with torch.no_grad():
    output_ids = model.generate(
        **inputs,
        max_new_tokens=60,
        do_sample=False, # greedy - deterministic
        pad_token_id=tokenizer.eos_token_id,
    )
elapsed = time.perf_counter() - start

# Step 5: Decode back to text
full_text = tokenizer.decode(output_ids[0], skip_special_tokens=True)
new_tokens = output_ids.shape[1] - inputs["input_ids"].shape[1]

print("\n=== Greedy Output ===")
print(full_text)
print(f"\nNew tokens : {new_tokens}")
print(f"Elapsed      : {elapsed:.2f}s ({new_tokens/elapsed:.1f} tok/s)")

```

2.4 Inspecting Model Architecture

```

from transformers import AutoModelForCausalLM
import torch

model = AutoModelForCausalLM.from_pretrained("gpt2")

# Count parameters
total_params = sum(p.numel() for p in model.parameters())
trainable    = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total parameters      : {total_params:,}")
print(f"Trainable parameters : {trainable:,}")

```

```
# Estimate RAM footprint
fp32_gb = total_params * 4 / 1e9
fp16_gb = total_params * 2 / 1e9
print(f"\nFP32 RAM estimate : {fp32_gb:.2f} GB")
print(f"FP16 RAM estimate : {fp16_gb:.2f} GB")

# Print architecture overview
print("\nModel architecture:")
print(model)
```

3. Testing & Metrics

3.1 Latency Benchmark Script

```
import time, statistics
from transformers import pipeline

gen = pipeline("text-generation", model="gpt2", device=-1)
prompt = "Once upon a time in a land of large language models"

latencies = []
for i in range(5):
    t0 = time.perf_counter()
    out = gen(prompt, max_new_tokens=50, do_sample=False)
    latencies.append(time.perf_counter() - t0)
    print(f"Run {i+1}: {latencies[-1]:.3f}s")

print(f"\nMean      : {statistics.mean(latencies):.3f}s")
print(f"Std dev   : {statistics.stdev(latencies):.3f}s")
print(f"Min       : {min(latencies):.3f}s")
print(f"Max       : {max(latencies):.3f}s")
```

3.2 Memory Profiling with tracemalloc

```
import tracemalloc
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

tracemalloc.start()
snap1 = tracemalloc.take_snapshot()

model = AutoModelForCausalLM.from_pretrained("gpt2")
tokenizer = AutoTokenizer.from_pretrained("gpt2")

snap2 = tracemalloc.take_snapshot()
stats = snap2.compare_to(snap1, "lineno")
total_mb = sum(s.size_diff for s in stats) / 1e6
print(f"RAM used by model load: {total_mb:.1f} MB")
```

3.3 What to Look For

Metric	Target (GPT-2 on CPU)	How to Measure
--------	-----------------------	----------------

Token generation speed	5–20 tok/s	new_tokens / elapsed
Model load time	2–8 s	time.perf_counter() around from_pretrained()
RAM footprint (GPT-2)	~500 MB	tracemalloc or /usr/bin/time -v
Output coherence	Grammatically fluent	Manual inspection

3.4 Quick Sanity Checks (Shell)

```
# Verify install
python -c "import transformers; print(transformers.__version__)"
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"

# One-liner generation test
python -c "
from transformers import pipeline
g = pipeline('text-generation', model='gpt2')
print(g('AI is', max_new_tokens=20)[0]['generated_text'])
"

# Check model cache location
python -c "from transformers import file_utils; print(file_utils.default_cache_path)"

# Monitor RAM during load (Linux/Mac)
/usr/bin/time -v python -c "
from transformers import AutoModelForCausalLM
AutoModelForCausalLM.from_pretrained('gpt2')
" 2>&1 | grep "Maximum resident"
```

Key Takeaways

- **pipeline()** is the fastest path to a running inference — one object, one call.
- **AutoModel + AutoTokenizer** gives you raw tensors, essential for custom training loops, batching, and advanced decoding.
- Autoregressive generation is sequential — every new token requires a full forward pass.
- RAM consumption scales linearly with parameter count × dtype bytes.
- Always benchmark both latency AND memory before moving to larger models.

Next topic: **Ollama** — install & run LLaMA 3 via CLI and REST API, then compare responses against HuggingFace.